UNIT-IV

Audio Event Identification and Classification

Introduction

- Creating pretrained deep learning models for audio data is a huge challenge
- For image data – efficient pretrained models – VGG/Inception
- For text data – efficient pretrained word embedding based models – GloVe / Word2vec
- Aspects to be covered
  1. Understanding audio event classification
  2. Formulating our real world problems
  3. Exploratory analysis of audio events
  4. Feature Engineering and representation of audio events
  5. Audio event classification with transfer learning
  6. Building a deep learning audio event identifier

1. Understanding audio event classification

- An audio event is basically an event or occurrence of an activity that is usually captured by an audio signal.
- Usually, short audio clips are used to represent audio events since, even if they are recurring, the sounds are usually similar.
- Sometimes, longer audio clips might be used to represent more complex audio events.
- Eg:childing playing in a playground, a siren alarm, Dogs barking.
- Google has built a massive dataset called AudioSet which is a large scale dataset of manually annotated audio events (over 632 audio event classes consisting of a collection of 2,084,320 human-labelled 10-second sound clips drawn from YouTube videos)

2. Formulating our real-world problem

- Objective
  - Audio event identification and classification
  - Supervised learning problem where we will be working on an audio event dataset with samples of audio data that belong to specific categories.
  - Dataset
    - UrbanSound8K dataset and has 8,732 labeled audio sound files (duration of >=4 seconds) that contain excerpts from common urban sounds.
    - 10 categories of sounds
      - Air_conditioner
      - Car_horn
      - Children_playing
      - Dog_bark
      - Drilling
      - engine-_idling
      - Gun_shot
      - Jackhammer
      - Siren
      - Streen_music

3. Exploratory analysis of audio events

- Once all the data is downloaded, then there are 10 folders containing audio data samples in WAV format.
- We also have metadata folder which contains metadata information for each audio file in the UrbanSound8K.csv file
- Each audio file is named in a specific format.
- The name takes the [fsID]-[classID]-[occurrenceID]-[sliceID].wav format which is populated as
  - [fsID]-The freesound ID of the recording from which this excerpt (slice) is taken
  - [classID]-A numeric identifier of the sound class
  - [occurrence ID] – A numeric identifier to distinguish different occurrences of the sound within the original recording.
  - [sliceID] – A numeric identifier to distinguish different slices taken from the same occurrence

Exploratory analysis of audio events

- Each class identifier is a numeric number that can be mapped to a specific class label.
- Exploratory analysis
  - Load up the dependencies including librosa module
  - The librosa module is an excellent open source Python framework for audio and music analysis.
  - It provides various functions to quickly extract key audio features and metrics from audio files.
  - Librosa can be used to analyze and manipulate audio files in a variety of formats such as WAV, OGG, MP3, FLAC, etc.

Exploratory analysis of audio events – Load dependencies

```
import glob
import os
import librosa
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.pyplot import specgram
import pandas as pd
 import librosa.display
import IPython.display
import soundfile as sf
%matplotlib inline
```

Exploratory analysis of audio events

- Load one folder of data for analysis

```
files = glob.glob('UrbanSound8K/audio/fold1/*')
len(files)
873
```

- Each folder roughly contains 870+ audio samples.

Exploratory analysis of audio events

- Based on the information from the metadata and readme files, we can now create a classID for name mapping the audio sample categories.

```
class_map= {'0' : 'air_conditioner', '1' : 'car_horn', '2' :
'children_playing', '3' : 'dog_bark', '4' : 'drilling', '5' : 'engine_idling', '6' :
'gun_shot', '7' : 'jackhammer', '8' : 'siren', '9' : 'street_music'}
```

```
pd.DataFrame(((class_map.items())))
```

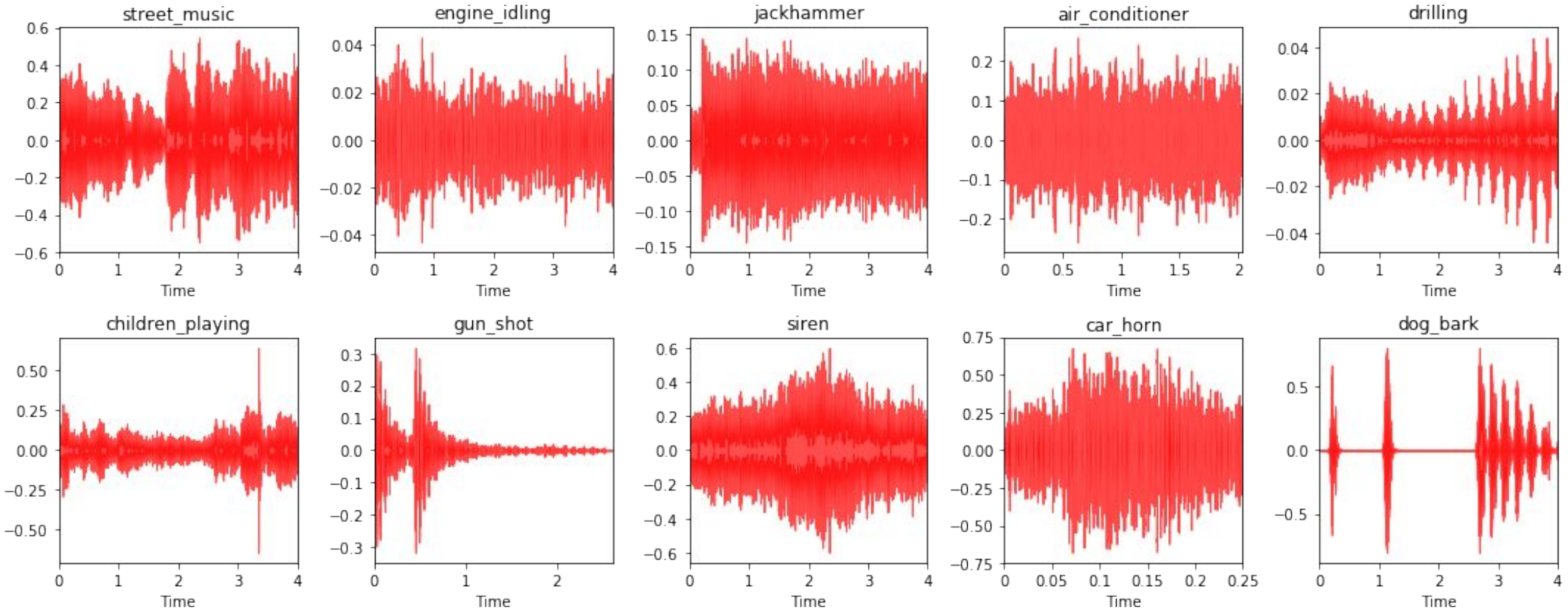
Exploratory analysis of audio events

- Let us take ten different audio samples belonging to each of these classes now, for further analysis.
- Now that we have our sample data files, we still need to read in the audio data into memory before we can do any analysis.
- Librosa was throwing an error for some of the audio files due to short length or sample rates.
- Hence, we leverage the sound file python framework to read in the audio files to get their raw data and their original sample rate.
- Audio sample rate is defined as the number of samples of audio carried per second, which is usually measured in Hz or kHz.
- The default sampling rate for librosa is 22,050 Hz, which is what we will be re-sampling all our audio data into to maintain consistency.

Exploratory analysis of audio events-Visualization technique

- In Jupyter Notebook, we can even embed the audio in the notebook itself and play it using the snippet
- Ipython,display.display(Ipython.display.Audio(data=data[1[0],rate=data[1][1]))
- Let us now visualize what these different audio sources look like by plotting their waveforms.
- Typically this will be a waveform amplitude plot for each audio sample
- Code snippet – refer **Exploratory Analysis Sound Data.ipynb file**

Exploratory analysis of audio events - Waveform amplitude plot for each audio sample

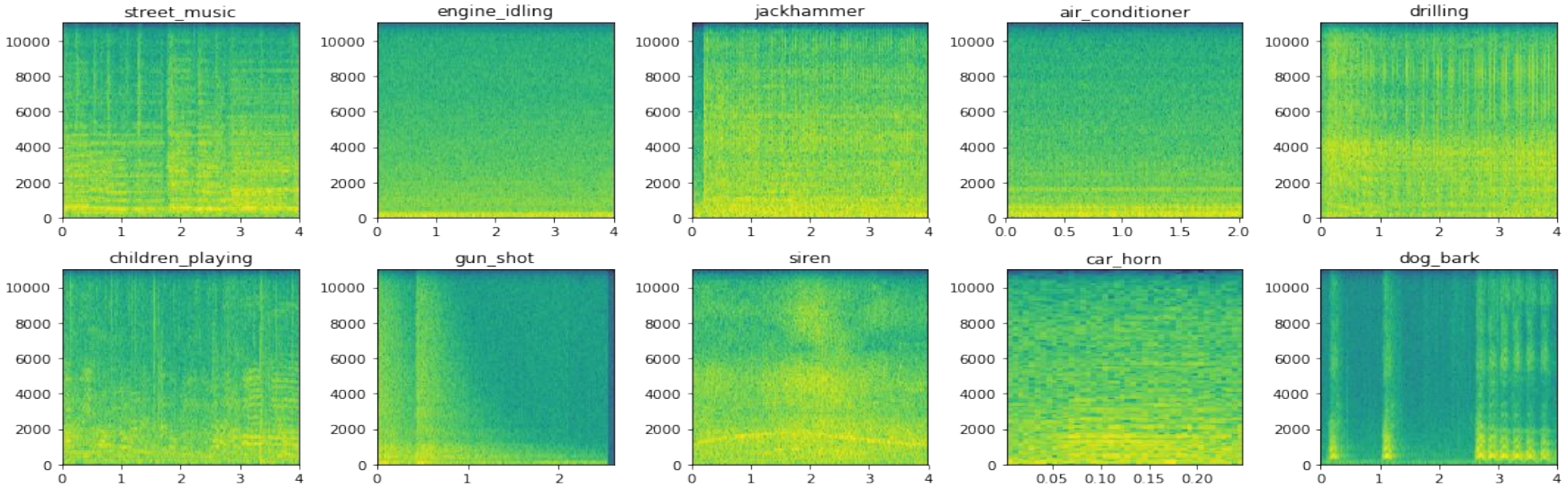


Exploratory analysis of audio events

- Interpretation
  - Sources like engine-idling,jackhammer,air_conditioner have a constant sound which does not change over time. Hence we can notice constant amplitude in the waveforms.
  - Siren and car_horn have a constant audio waveform with intermittent increases of amplitude.
  - The gun_shot usually have a huge noise in the beginning followed by silence
  - The dog_bark comes in intermittent intervals.

Exploratory analysis of audio events – visualization technique - spectrograms

- A spectrogram is a visualization technique for representing spectrums of frequencies from audio data.
- Also known as sonographs and voicegrams.
- Code snippet – refer **Exploratory Analysis Sound Data.ipynb file**

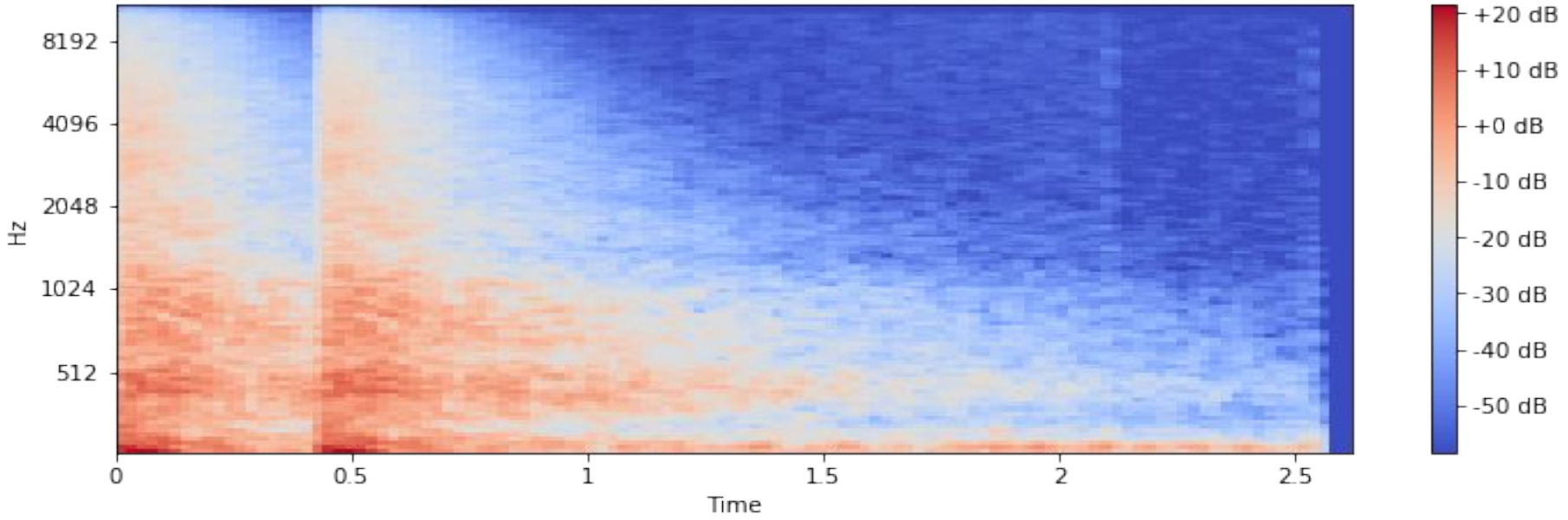


Exploratory analysis of audio events

- Here mel spectrogram is used which is usually better than a basic spectrogram because it represents the spectrogram on a mel-scale.
- The name mel comes from the word melody.
- This indicates that the scale is based on pitch comparisons
- The mel-scale is thus a perpetual scale of pitches that have been judged by listeners to be equal in distance from one another
- This is useful when we use CNN to extract features from these spectrograms.
- Code snippet – refer **Exploratory Analysis Sound Data.ipynb file**

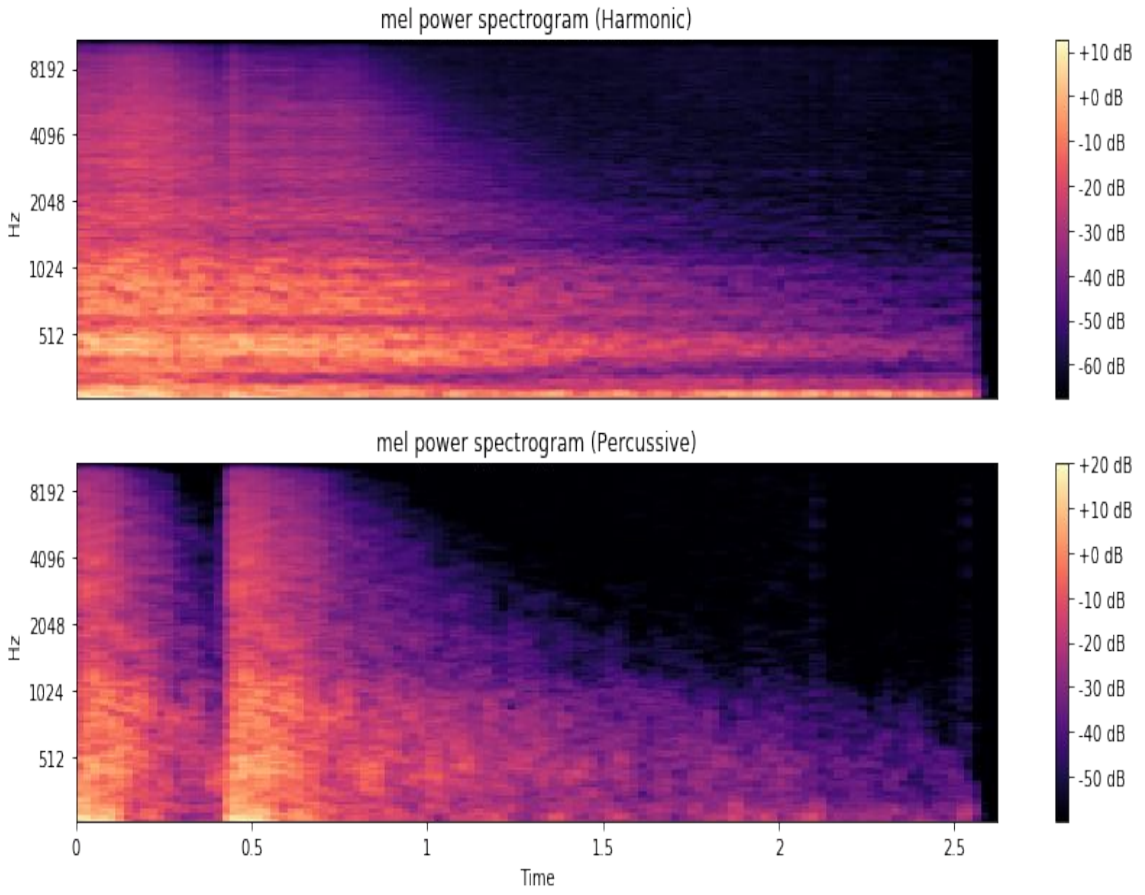
Exploratory analysis of audio events – visualization technique – mel spectrograms

- With mel-scale, the spectrograms are much easier to distinguish based on the audio sources.
- It will be used for feature engineering
- Eg: Consider gun_shot audio sample with mel spectrogram



Exploratory analysis of audio events

- The spectrogram is consistent with how the audio waveform is for the audio source.
- Usually any audio time series data can be decomposed into harmonic and percussive components.
- These can present completely new and interesting representations of any audio sample.
- Code snippet – refer **Exploratory Analysis Sound Data.ipynb file**

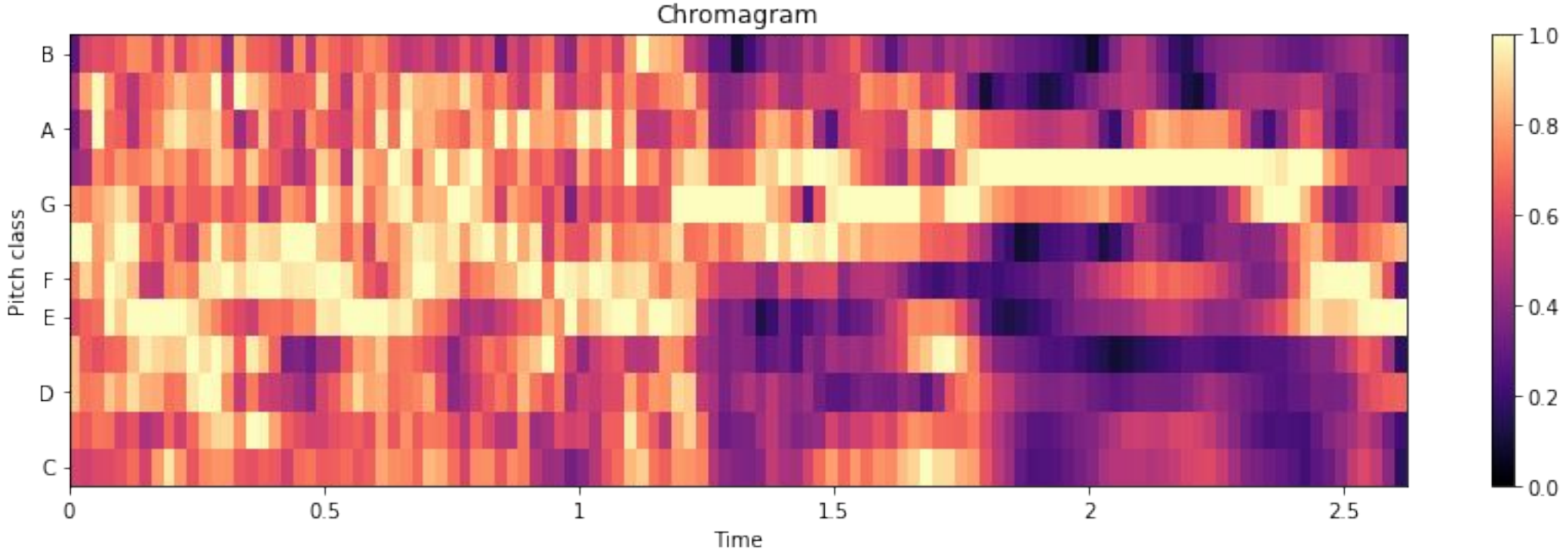


Exploratory analysis of audio events – visualization technique - Chromagram

- Chromagram shows the pitch intensities of the audio sample signal based on 12 different pitch classes namely [C,C#,D,D#,E,F,F#,G,G#,A,A# and B].
- This can be an excellent visual tool for depicting the various pitch intensities in the audio signal over time.
- Typically a Fourier transform or Q-transform is done on the raw audio signal before building a chromagram.
- Code snippet – refer **Exploratory Analysis Sound Data.ipynb file**

Exploratory analysis of audio events – visualization technique - Chromagram

- We can clearly see the various pitch intensities of gun_shot audio sample over time, which will definitely be effective as a base image for feature extraction.



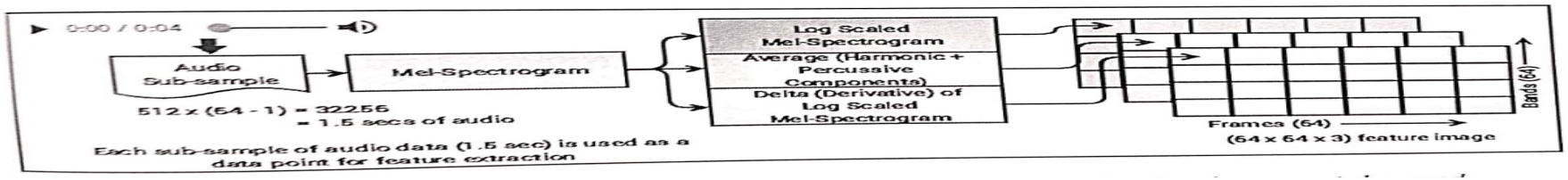
4. Feature Engineering and Representation of Audio Events

- To save features to disk

  from sklearn.externals import joblib
- **Get all filenames**
- **Load raw audio data**
- **Function to get start and end indices for audio sub-sample**

Refer Feature Engineering.ipynb

4. Feature Engineering and Representation of Audio Events



Feature Engineering and Representation of Audio Events

- Since audio data samples are of varying length, in order to build robust classifier, our features need to be consistent per sample.
- Hence we will be extracting audio sub-samples of fixed length from each audio file and extracting features from each of these sub-samples.
- We will be using a total of three feature engineering techniques to build three features representation maps, which will ultimately give us a 3D image feature map for each of our audio sub-samples.

Feature Engineering and Representation of Audio Events - Workflow

- Mel-spectrograms are leveraged to general necessary features that can be consumed by CNNs for feature extraction.
- Step1: Define the total number of frames (columns) to be 64 and bands (rows) to be 64, which forms the dimensions of each of our feature maps (64x64). Based on this, we extract windows of audio data, forming sub-samples from each audio data sample.
- Step 2: From each audio sub-sample, mel spectrogram is created. From this, we create a log-scaled mel spectrogram as one of the feature maps, an averaged feature map of the harmonic and percussive components of our audio sub-sample and the delta or derivative of our log-scaled mel spectrogram as the third feature map. Each of these feature maps can be represented as a 64x64 image and by combining them we get a 3D feature map of dimensions(64,64,3) for each audio sub-sample.

Feature Engineering and Representation of Audio Events - Workflow

- Refer extract_features function in Feature Engineering.ipynb
- It wil be used on all 8,732 audio samples to create feature maps out of many sub-samples.

  features,labels=extract_features(files)

  features.shape,labels.shape

  ((30500,64,64,3).(30500,))
- We get a total of 30,500 feature maps from our 8,732 audio data files.
- Each feature map is of dimensions (64,64,3)

Feature Engineering and Representation of Audio Events - Workflow

- Overall class representation for our audio sources based on these 30,500 data points

  From collections import Counter

  Counter(labels)

  Counter({0:3993, 1:913, 2:3947, 3:2912, 4:3405, 5:3910, 6:336, 7:3473, 8:3611, 9: 4000})

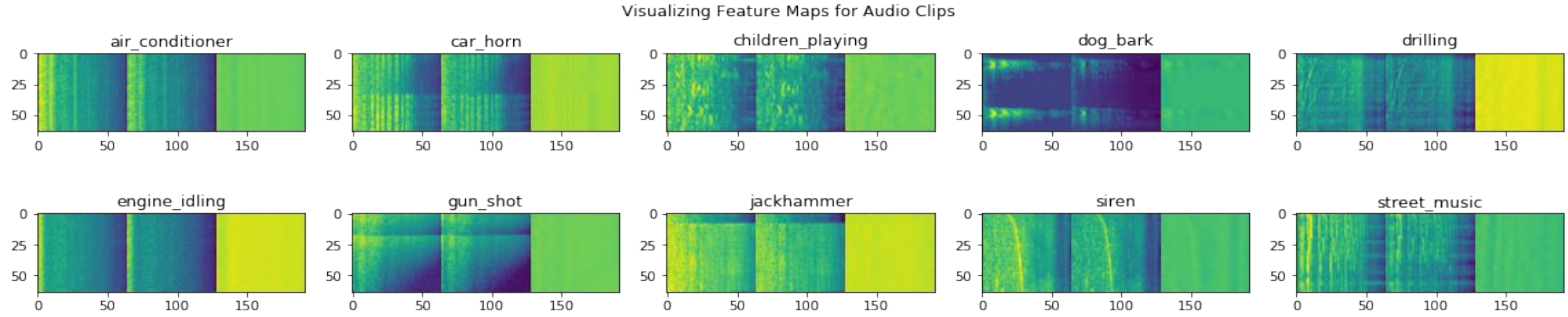
- From above representation, we can say that the overall distribution of data points in different categories is quite uniform and decent.
- For some categories like 1 (car_horn) and 6 (gun_shot), the representation is quiet low as compared to other categories.
- This is because, audio data duration for these categories is typically much shorter than with the other categories.

Feature Engineering and Representation of Audio Events – Visualization of feature maps

- Refer Feature Engineering.ipynb
- Feature maps will appear as follows
- One feature map for each category and each feature map is a 3D image
- Save these base features to disk

  joblib.dump(feaures,'base_features.pkl')

  joblib.dump(labels,dataset_labels.pkl')
- These base features will act as a starting point for further feature engineering



5. Audio event classification with transfer learning

- Here VGG-16 model is used as a feature extractor and then train a fully-connected deep network (MLP) on these features.
- Steps
  1. Building datasets from base features
  2. Transfer Learning for feature extraction
  3. Building the classification model
  4. Evaluating the classifier performance

5. Audio event classification with transfer learning

- 1. Building datasets from base features
  - Load the base features and create train, validation and test datasets.
  - Randomly shuffle the data and create train, validation and test datasets.

  Train: Counter({9: 2448, 2: 2423, 0: 2378, 5: 2366, 8: 2140, 7: 2033, 4: 2020, 3: 1753, 1: 542, 6: 197})

  Validate: Counter({0: 802, 5: 799, 2: 774, 9: 744, 8: 721, 7: 705, 4: 688, 3: 616, 1: 183, 6: 68})

  Test: Counter({0: 813, 9: 808, 2: 750, 8: 750, 5: 745, 7: 735, 4: 697, 3: 543, 1: 188, 6: 71})Check per class distribution in each of these datasets using counter.
- We can see a consistent and uniform distribution of data points per class across the datasets.

5. Audio event classification with transfer learning

2. Transfer Learning for feature extraction

- Now leverage transfer learning to extract useful features from our base feature map images for each data point.
- To do this, pretrained deep learning model which work very effective feature extractor on images – VGG-16 model is used.
- Load the VGG-16 model only as a feature extractor. Hence we will not be using its dense layers in the end.
- Input base feature map image has dimensions of (64,64,3) from which we will end up getting a 1D feature vector of size 2048.

5. Audio event classification with transfer learning

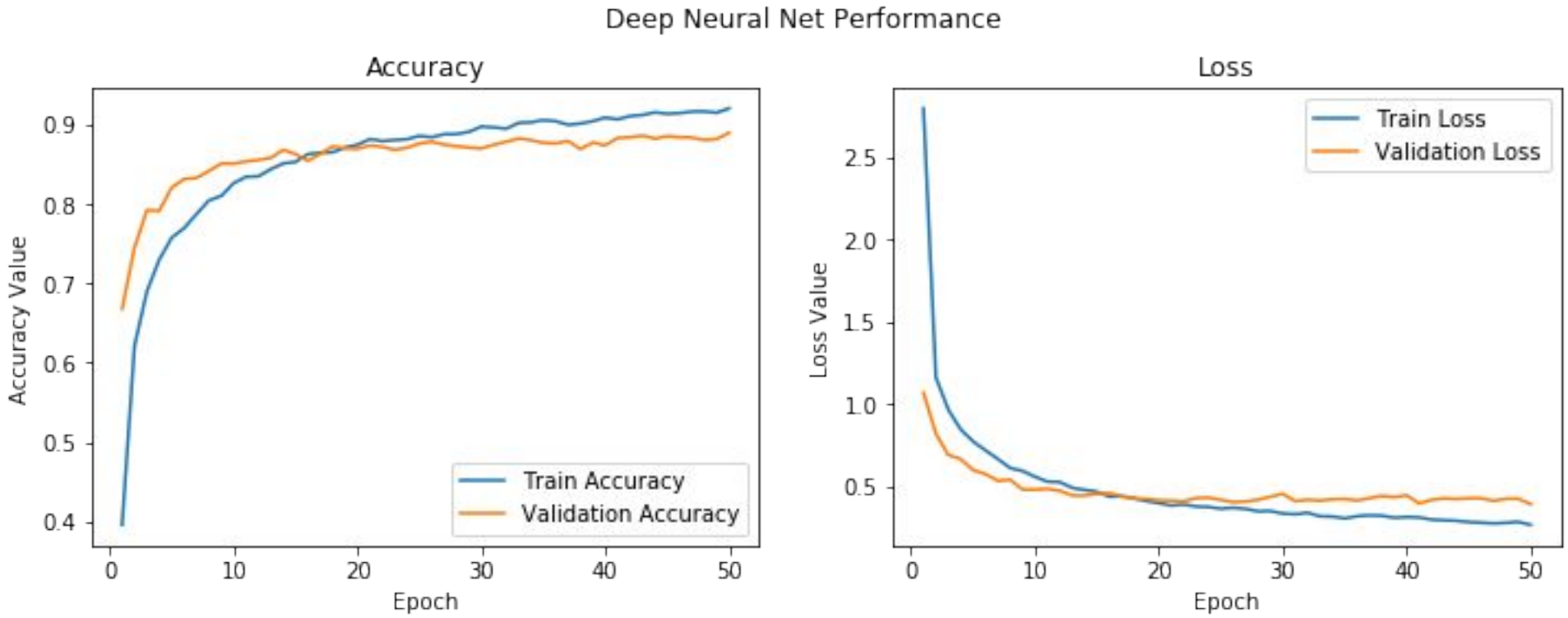
- Now build a generic function that will help us leverage transfer learning and get these features which are popularly known as **bottleneck features.**
- **Refer extract_tl_features function**
- This function can now be used along with our VGG-16 model to extract useful features from each of our audio sub-sample base feature map images. Do it for all our datasets.
  - Extract train dataset features
  - Extract validation dataset features
  - Extract test dataset features
- Now save these features and labels to disks so that we can use them later on, at any time for building our classifiers

5. Audio event classification with transfer learning

3. Building the classification model

- We are now ready to build our classification model on the features extracted.
- Refer **Modeling.ipynb**
- **Steps:**
- Load feature sets and data point class labels
- The input feature sets are 1D vectors of size 2048 obtained from VGG-16 model
- Now need to do one-hot encode the categorical class labels before we can feed them to a deep learning model.
- Now build our deep learning classifier by using fully connected network that has four hidden layers.
- Use dropout to prevent overfitting and Adam optimizer for our model.
- 50 epochs with batch size 128.
- Get validation accuracy of 89%

5. Audio event classification with transfer learning – Accuracy plot



Audio event classification with transfer learning

4. Evaluating the classifier performance

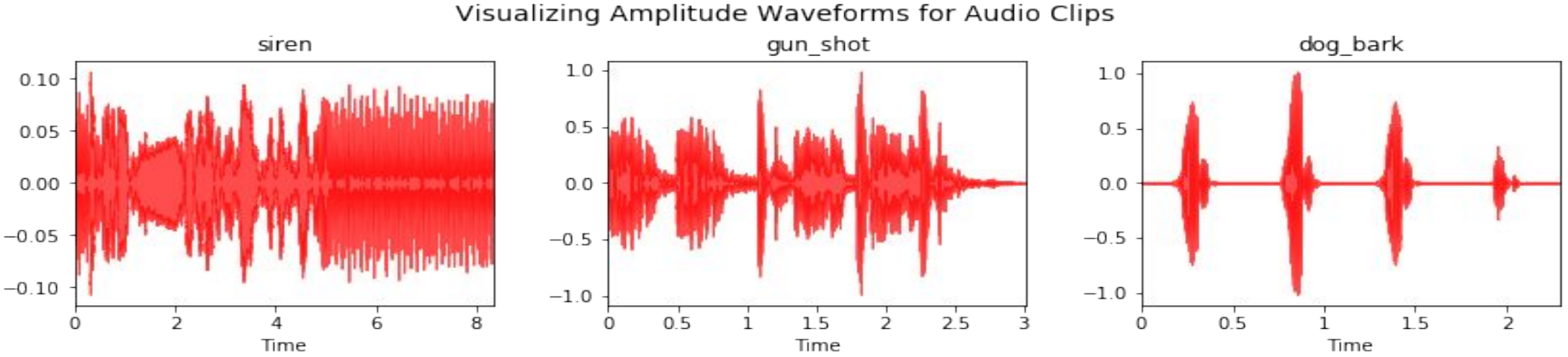
- We use the test dataset to make predictions with our model and then evaluate them against the ground truth labels.

Building a deep learning audio event identifier

- Here we are going to build an actual audio event identifier by leveraging the classification model we built in previous section
- This will enable us to take any new audio file and predict which category it might belong to by making use of the entire workflow starting from building the base feature maps, extracting features using VGG-16 model, and then leveraging our classification model to make a prediction.
- Refer **Prediction Pipeline.ipynb** file

Building a deep learning audio event identifier

- Initialize an instance of our class by feeding it the path of our classification model
- Download three completely new audio data files belonging to three of the ten audio classes and load them.
- Visualize waveforms of these three audio files and understand how they are structured.



Building a deep learning audio event identifier

- Based on the visualization, they seem consistent based on the audio source and our pipeline is working well.
- Now extract the base feature maps for these audio files. It is appeared as